

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #13,
CS@TUCN

Computer Science

Agenda

- Hamiltonian cycle – somehow NP complete/hard problems can be addressed
- Answering your questions & typical errors

Hamiltonian cycle with branch and bound

- Branch and bound – oral review
- Estimate the cost of the path in 2 steps, as follows:
 - $\varphi I = \psi i + X_i$
 - φI = cost of the path from Start to Stop via the *intermediate* node
 - ψi = cost of the path from Start to the *intermediate* node
 - X_i = cost of the path from the *intermediate* node to Stop
 - They are estimated as: $E\varphi i = E\psi i + EX_i$
 - $E\varphi I$ = estimate cost of the path from Start to Stop via the *intermediate* node
 - $E\psi I$ = estimate cost of the path from Start to the *intermediate* node
 - EX_i = estimate cost of the path from the *intermediate* node to Stop
- $E\psi I = \psi I$
- EX_i = the better the estimate, more we narrow the search space (bound more effective). Even a rough estimate could help. Even just 0 proves good.
- The approach (similar to `bf_search`) is also using 2 argument lists:
 - Candidate list – potential path to follow (it is a regular list; not incomplete)
 - Expanded list – part of paths already covered (from start to current node; none complete to objective)

Hamiltonian cycle with branch and bound - contd

- An element in the Cand list looks like: $[n(X, F_i, \text{Length}) | L_x]$
 X = node explored
 F_i = estimate value of φ_i $E\varphi_i = E\psi_i + EX_i = \psi_i + 0$
 Length = number of nodes so far, from Start to current (X)
 L_x = parent node = actually the whole path from parent to Start, backwards. It is a list!
- Helper predicates:
`eqlength(Lg, [n(_, _, Lg) | _]) .` % **Accesses** the **length** of an element in the Candidate list

`eqn(X, n(X, _, _)) .` % **Accessing** the **node**

`ord([n(_, Xfi, _) | _], [n(_, Yfi, _) | _]) :- Xfi < Yfi .` % Compares the $E\psi_i$ functions for nodes X and Y

`member_thread(X, [H | _]) :-`
`eqn(X, H) .` % checks if a node is in a list
`member_thread(X, [_ | T]) :-`
`member_thread(X, T) .`

`ins_ord_list(X, [H | T], [X, H | T]) :- ord(X, H) , ! .` % adds an element
`ins_ord_list(X, [H | T], [H | R]) :-` % in the right (ordered by F_i function) place in the list
`ins_ord_list(X, T, R) .`
`ins_ord_list(X, [], [X]) .` % REWRITE the insert predicate with just 2 clauses

Hamiltonian cycle with branch and bound - contd

```

hamilton(N,X,Way):-
    search(N,X,Way,[[n(X,0,0)]],[[]]).
% searches for N nodes to end in X the way; put in Cand a path containing just one element: [n(X,0,0)]
% start node X, estimated weight of ham EXi=0, number of nodes 0, NO parent. Expanded list empty.

search(N,X,Way,_,[Way|_]):-                % stop with path the way in front of Expanded
    eqlength(N,Way),!.                       % if its length equals the number of nodes in graph
search(N,X,Way,[Y|Cand],Exp):-
    expand(N,X,Y,Cand,NewCand), % if not, expand best node so far= the element in front of current Cand
    search(N,X,Way,NewCand,[Y|Exp]). % and continue after expansion with the new Cand

expand(N,X,[n(Y,Fi,Length)|Ly],_,_):-
    is_edge(Y,Z,W), % nondeterministically take Z, first neighbor of Y (eventually all of them) and weight W
    (not(member_thread(Z,Ly));(Length is N-1,Z=X)), % should NOT be already processed
                                                    %just if is the source
    FiW is Fi+W,Length1 is Length+1, % estimate the new parameters
    assertz(desc(n(Z,FiW,Length1))), % put it in the akb
    fail. % backtrack to a new neighbor of Y
expand(_,_,L,Cand,NewCand):-
    assertz(desc(end)), % mark the end of akb
    collect(L,Cand,NewCand). % start collecting from akb and place in NewCand

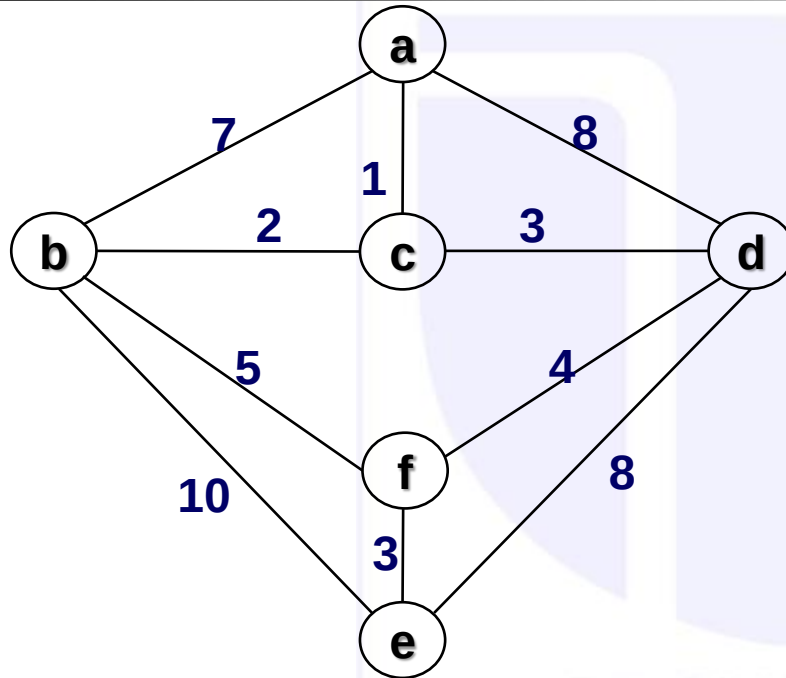
```

Hamiltonian cycle with branch and bound - contd

```
collect(L, Cand, NewCand) :- % L is the parent, a whole path. Is Y in Candidate from clause 2 in search
    get_next(Z), !, % take top from akb
    ins_ord_list([Z|L], Cand, IntCand), % add in Cand with the whole path (+L, parent of path)
    collect(L, IntCand, NewCand). % continue with the Intermediate Candidate
collect(_, Cand, Cand).

get_next(Z) :-
    retract(desc(Z)), !,
    Z \= end.
```

Example



Cand – ordered list of candidate partial paths, waiting expansion

Exp – accumulator list of expanded partial paths (also ordered)

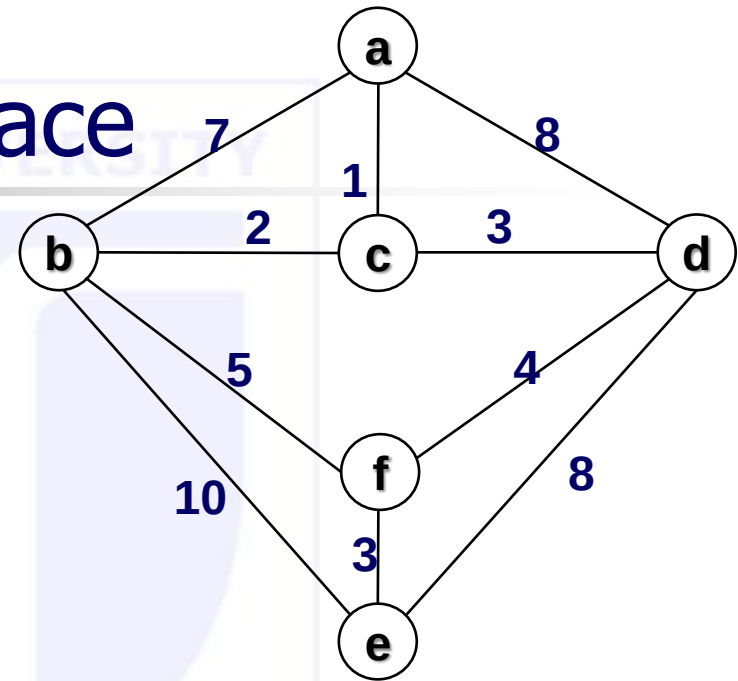
?- search(6,a,Way,[[n(a,0,0)],[]]).

result list; at the end (crt length = N) it is first element of **Exp**

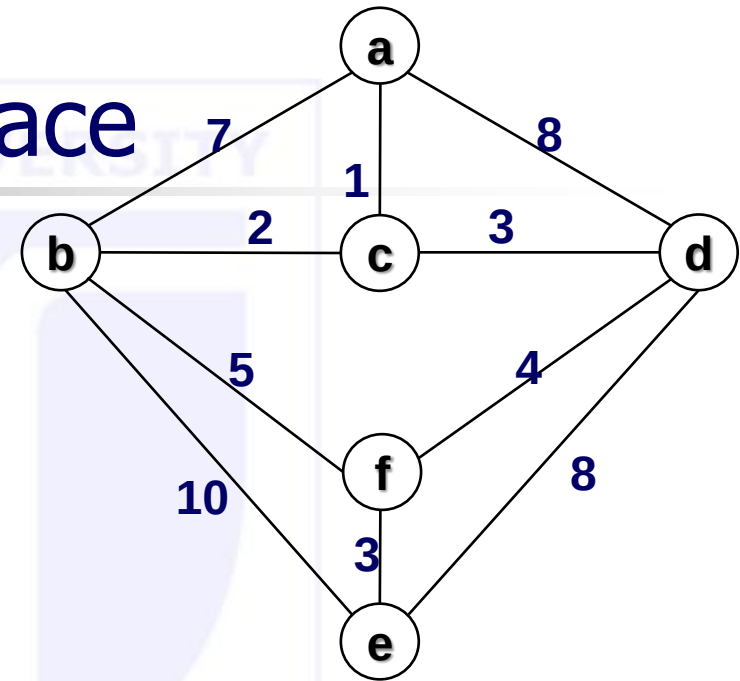
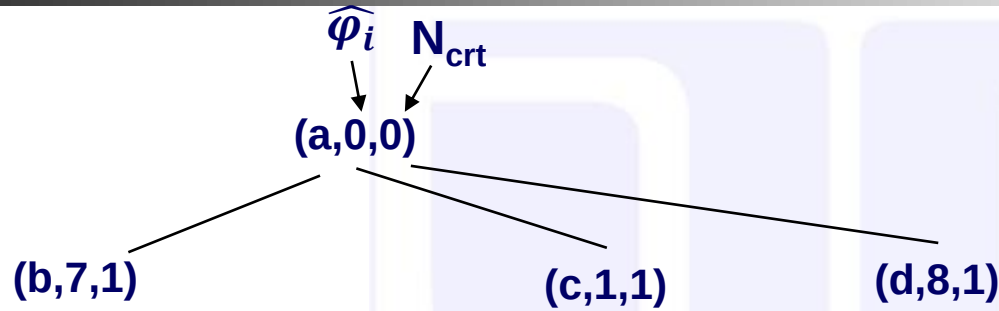
Example – search space

$\widehat{\varphi}_i$
 $(a,0,0)$

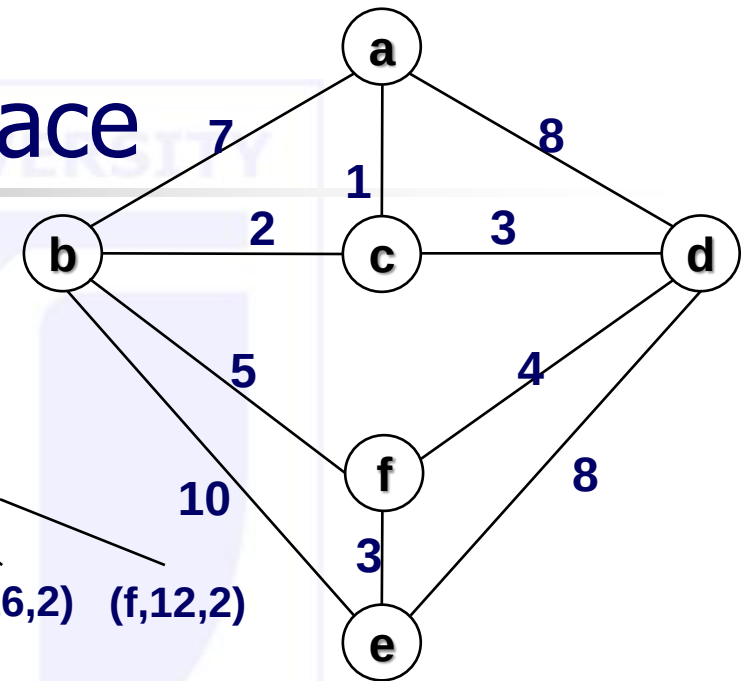
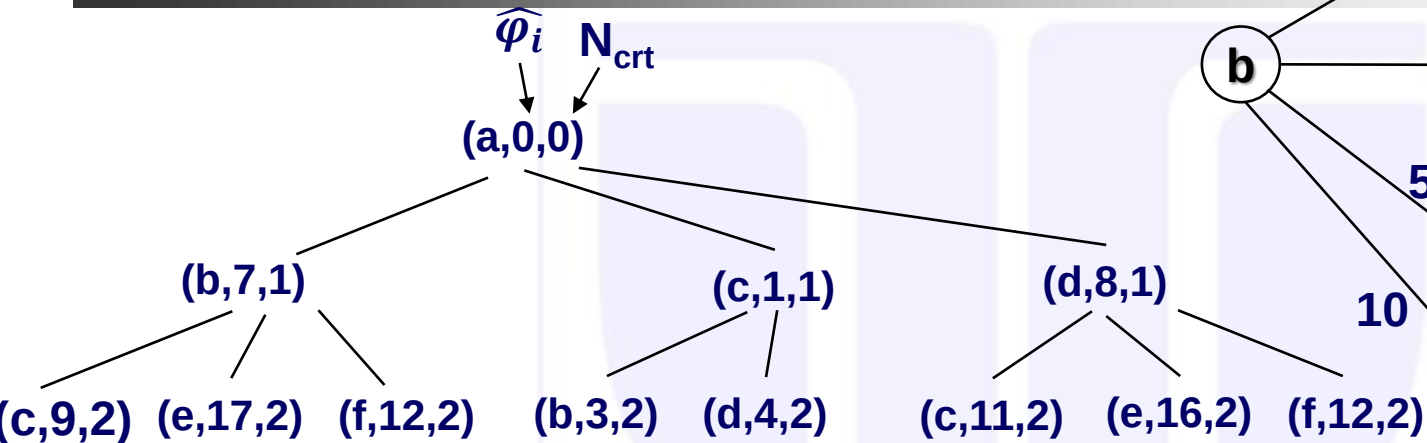
N_{crt}



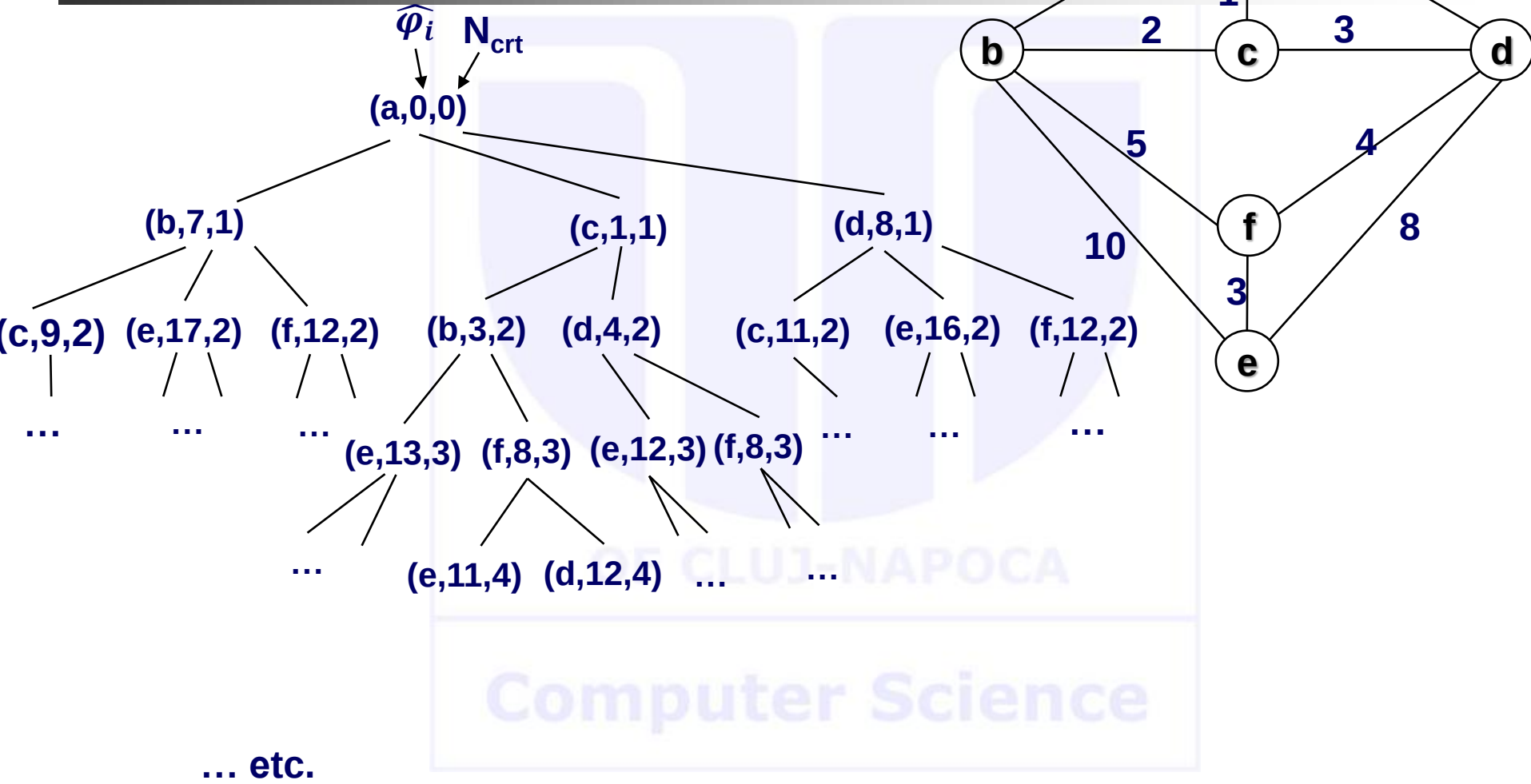
Example – search space



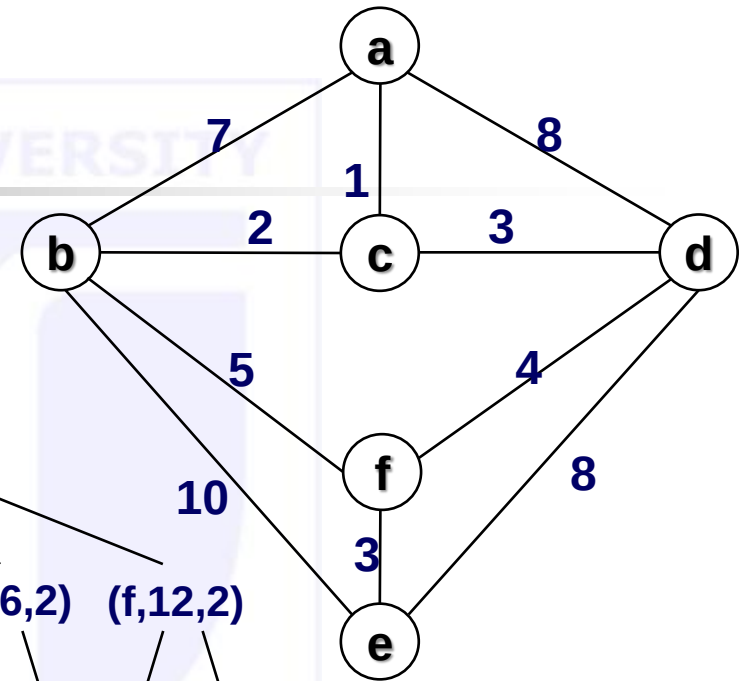
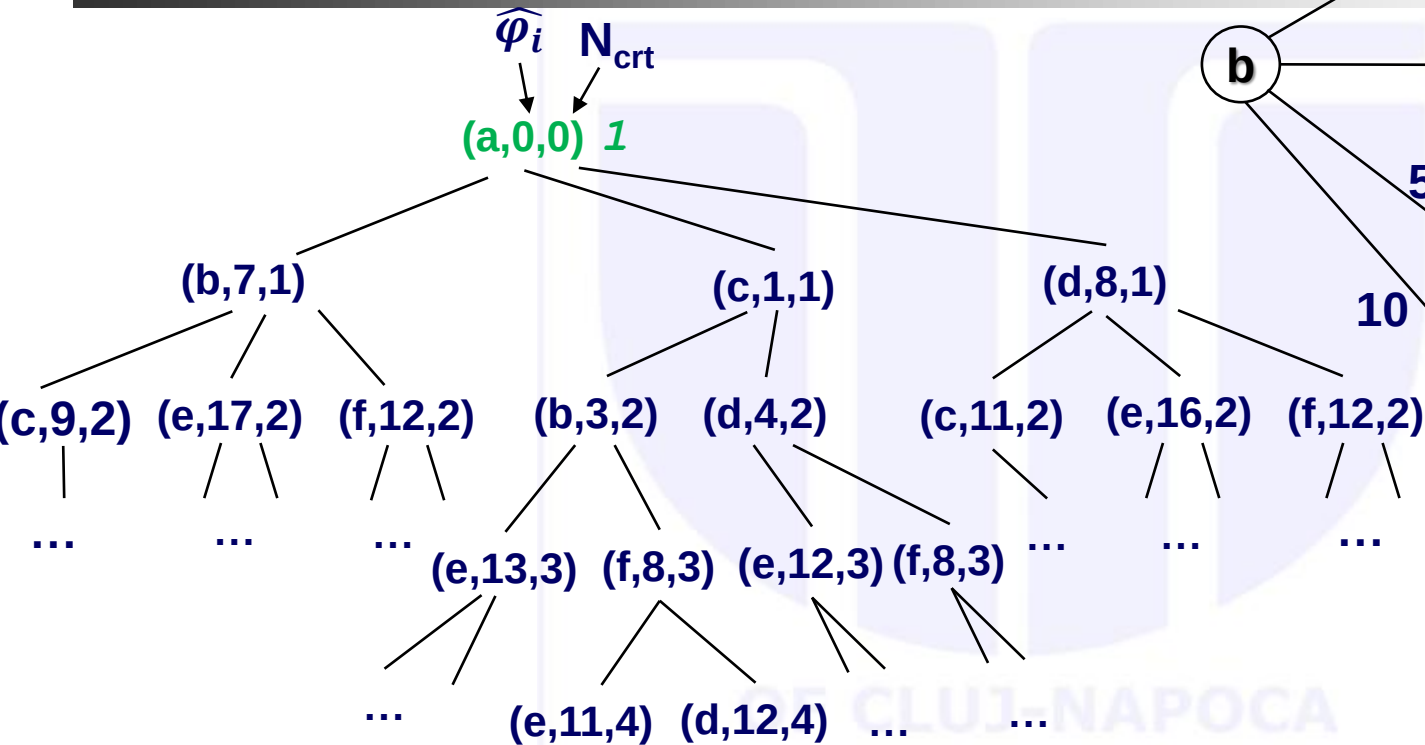
Example – search space



Example – search space



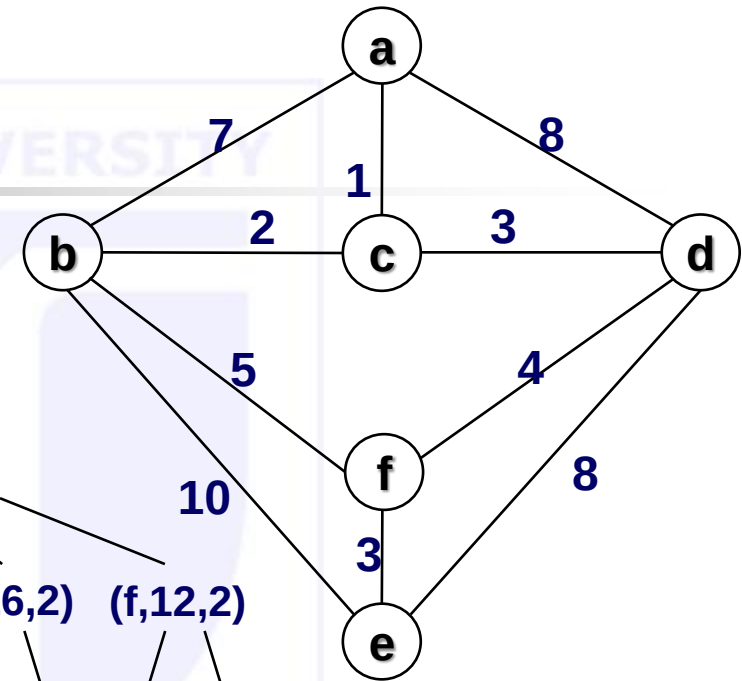
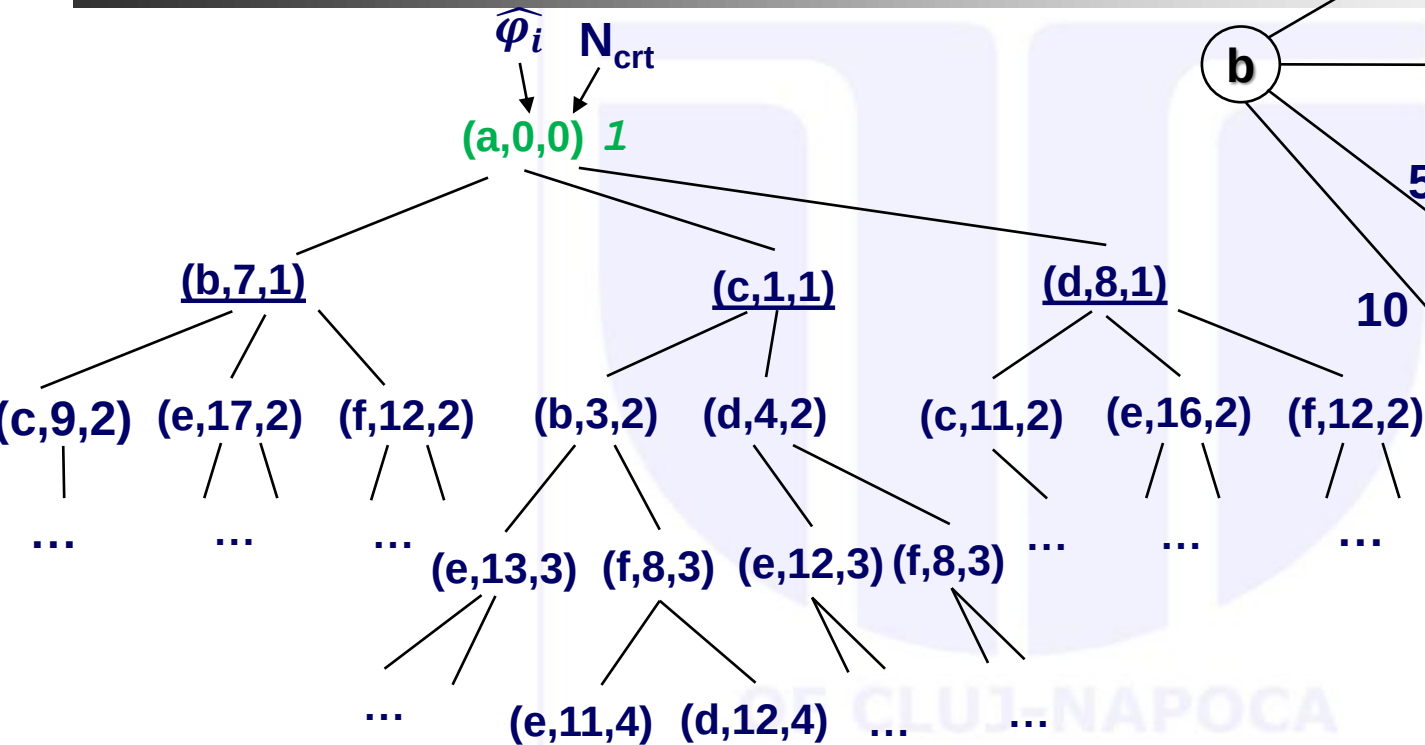
Example – traversal



Cand = $[[n(a,0,0)]]$
Exp = $[]$

... expand first element in Cand

Example – traversal

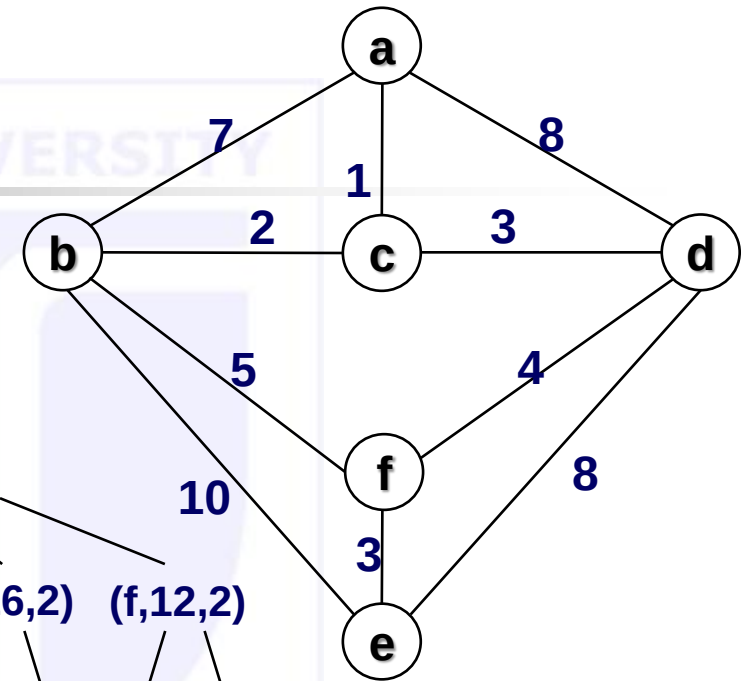
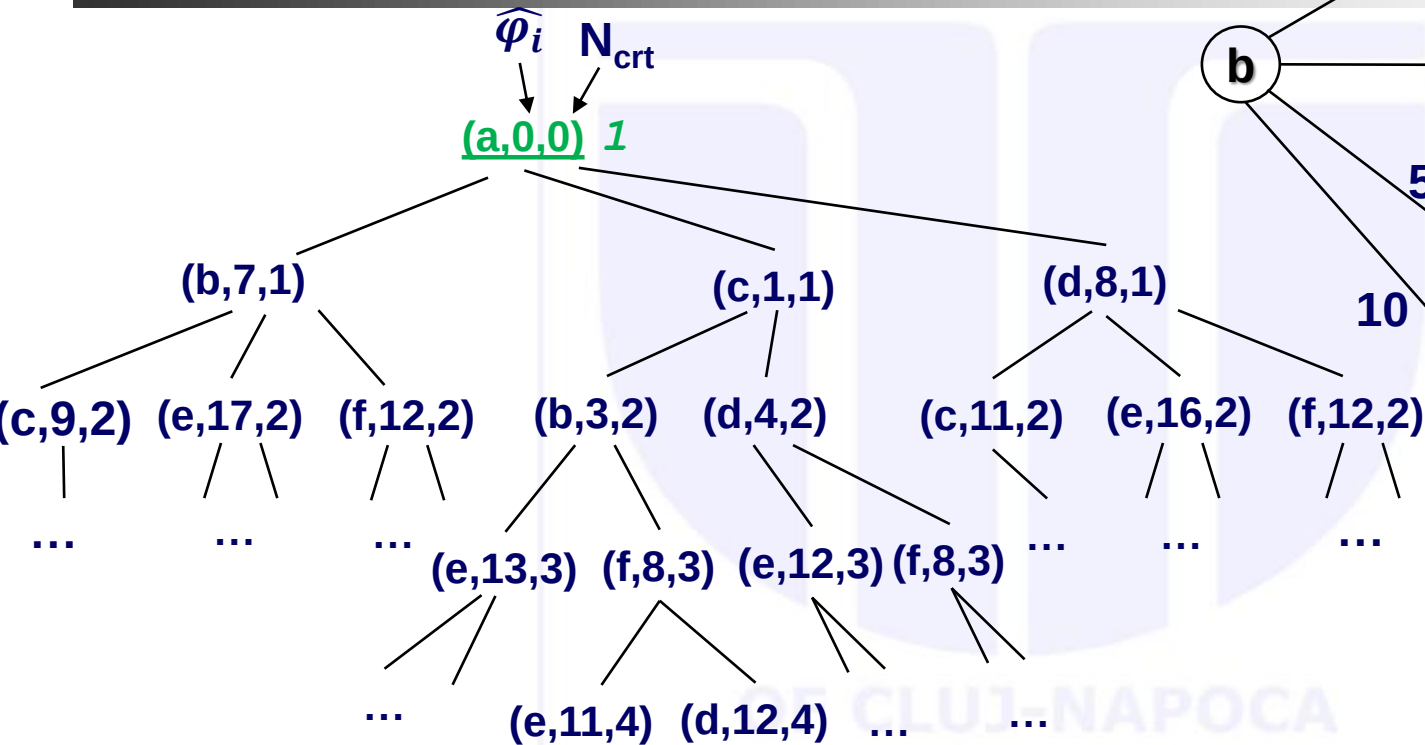


... after expanding $[n(a,0,0)]$

Cand = $[[n(c,1,1), n(a,0,0)], [n(b,7,1), n(a,0,0)], [n(d,8,1), n(a,0,0)]]$

Exp = $[]$

Example – traversal

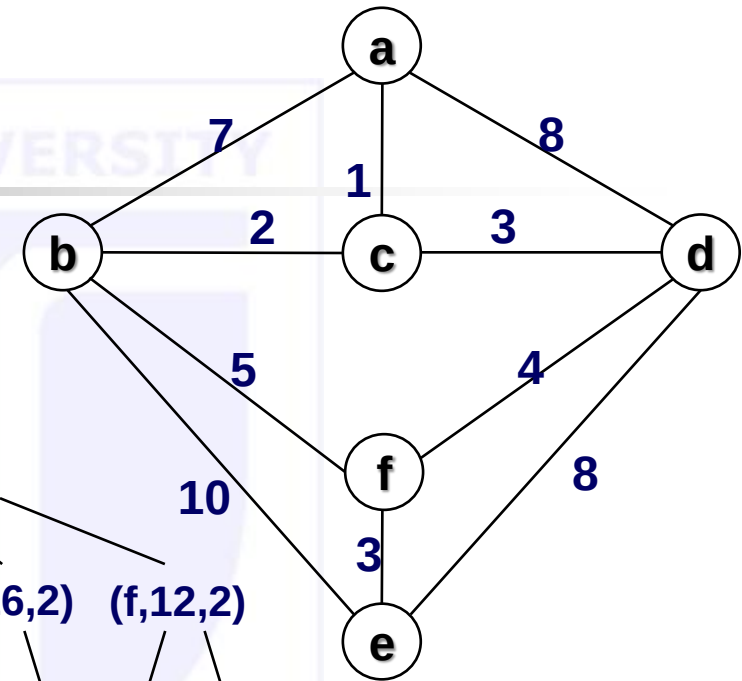
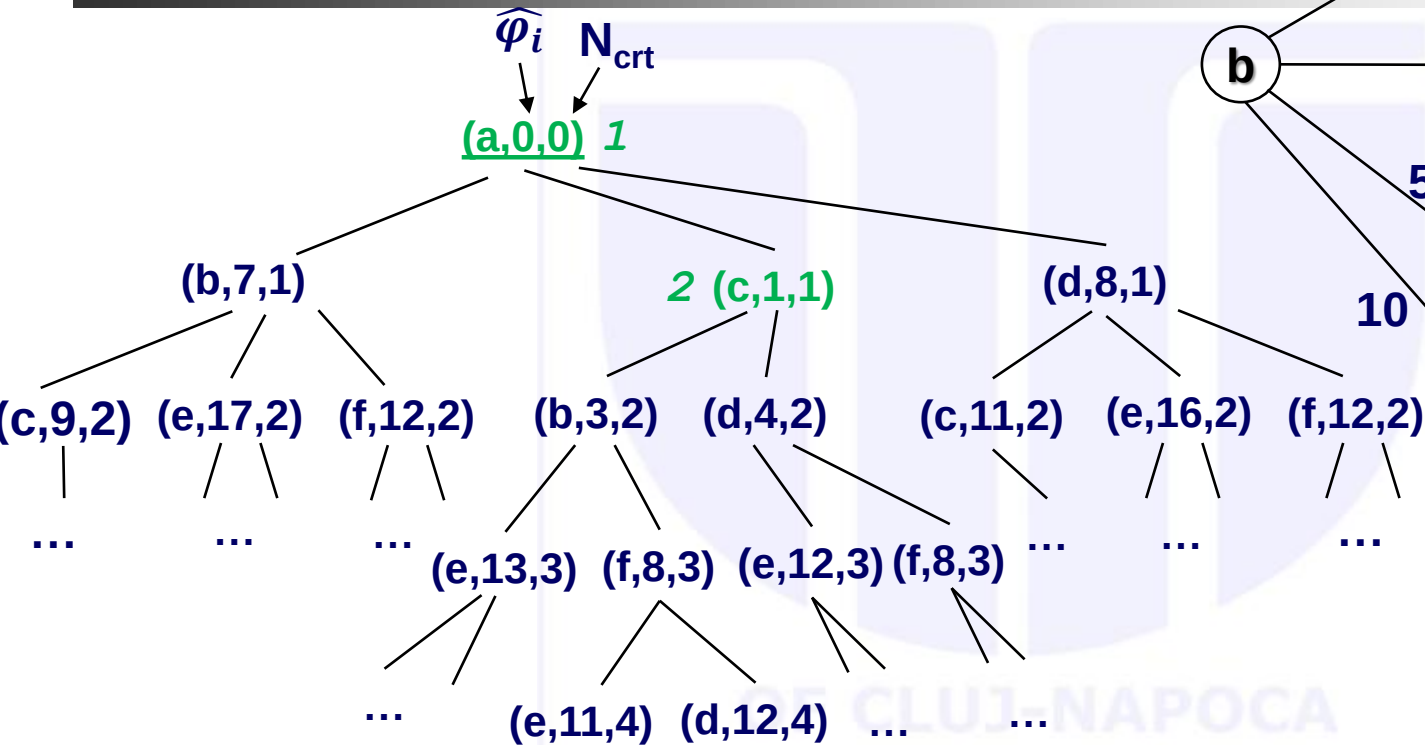


... recursive call search

Cand = $[[n(c, 1, 1), n(a, 0, 0)], [n(b, 7, 1), n(a, 0, 0)], [n(d, 8, 1), n(a, 0, 0)]]$

Exp = $[[n(a, 0, 0)]]$

Example – traversal

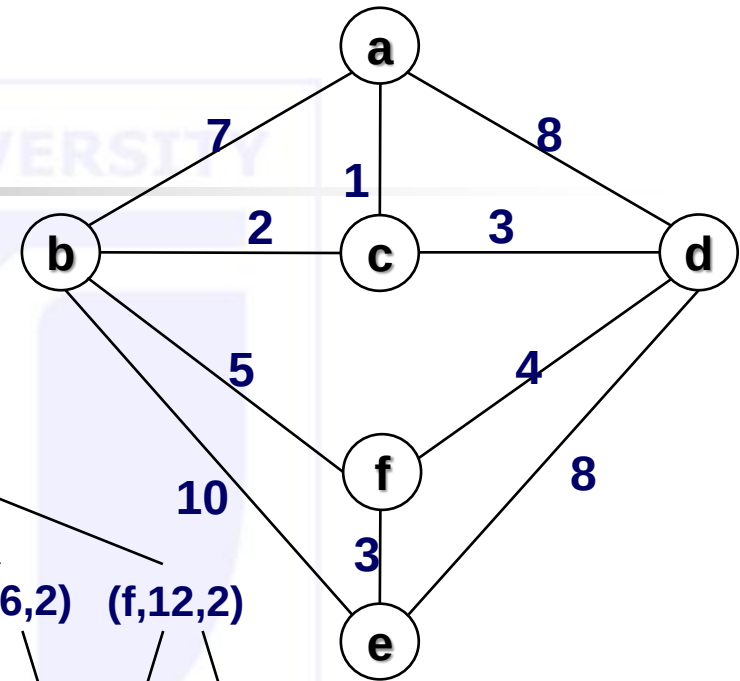
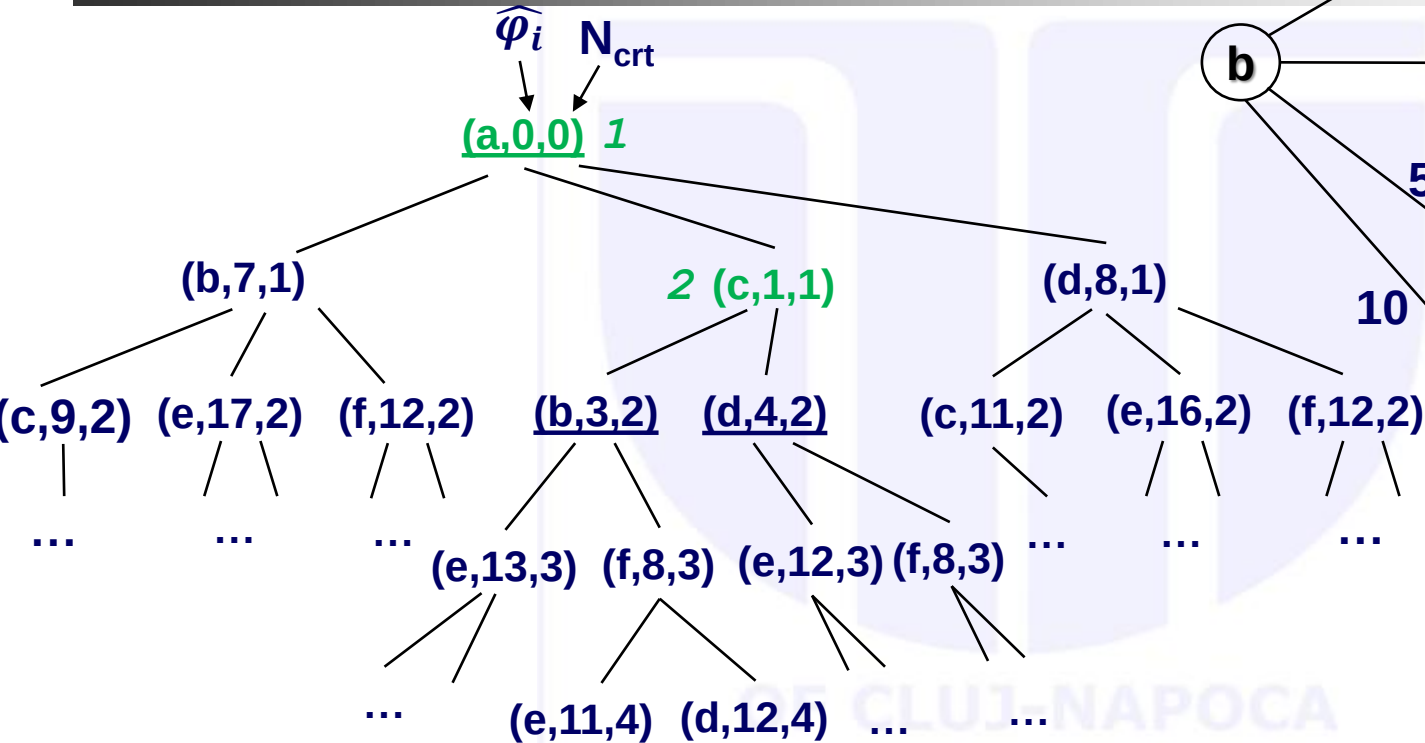


... this will be expanded next

Cand = $[[n(c,1,1), n(a,0,0)], [n(b,7,1), n(a,0,0)], [n(d,8,1), n(a,0,0)]]$

Exp = $[[n(a,0,0)]]$

Example – traversal

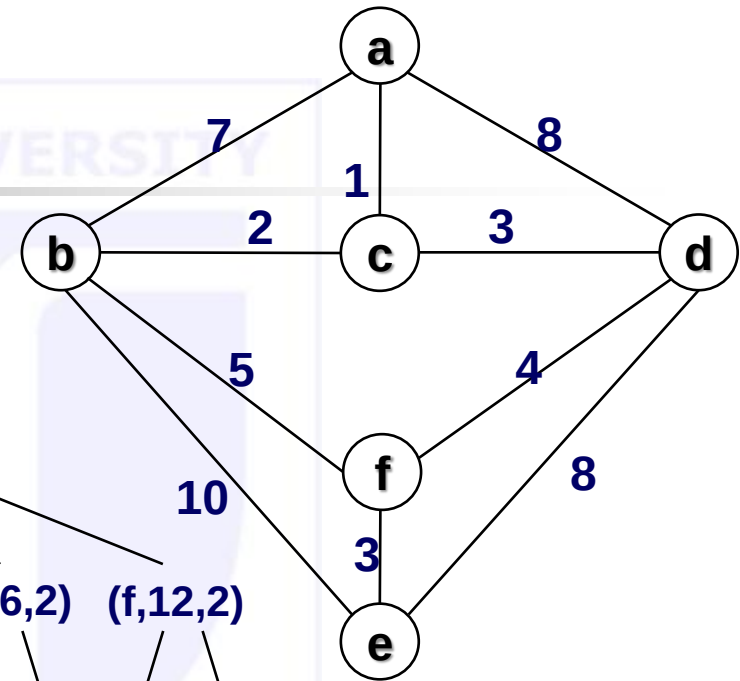
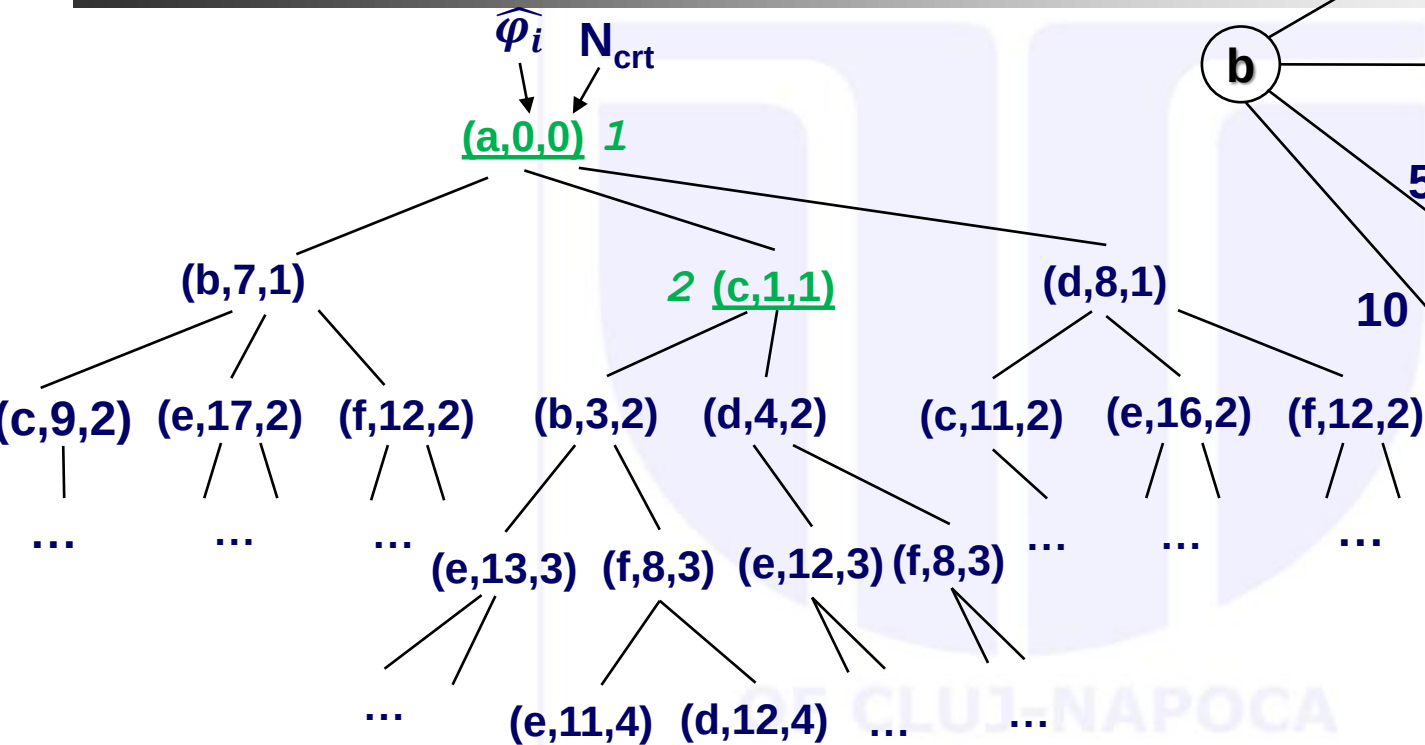


... after expanding $[n(c, 1, 1), n(a, 0, 0)]$

Cand = $[[n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)],$
 $[n(d, 4, 2), n(c, 1, 1), n(a, 0, 0)], [n(b, 7, 1), n(a, 0, 0)],$
 $[n(d, 8, 1), n(a, 0, 0)]]$

Exp = $[[n(a, 0, 0)]]$

Example – traversal

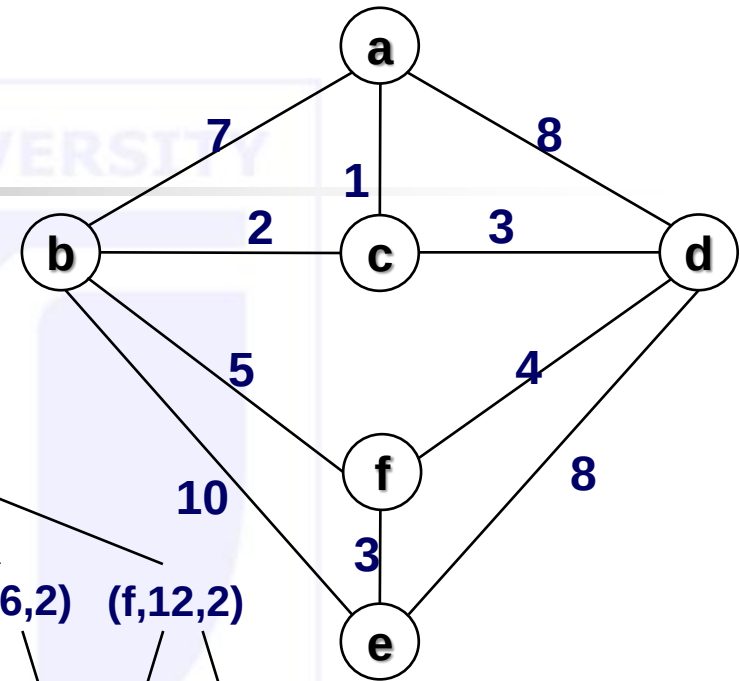
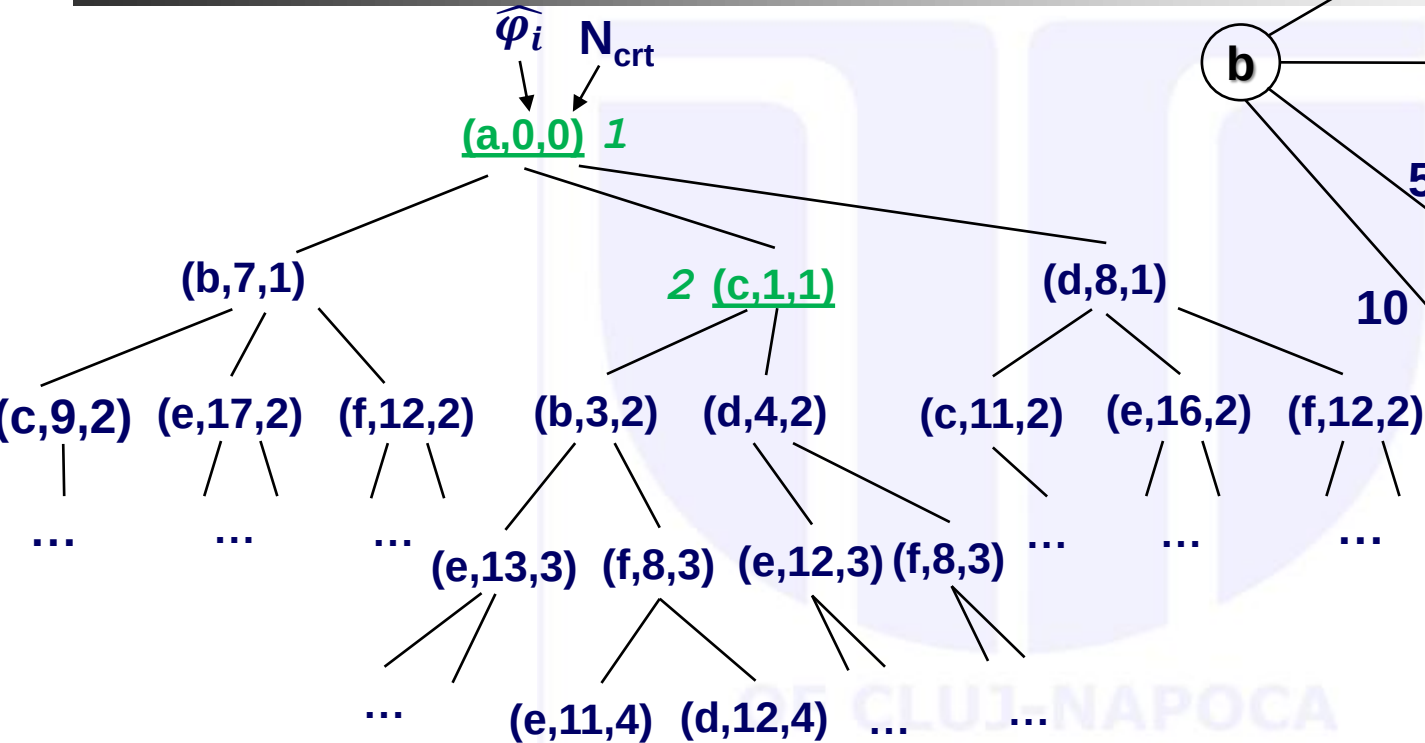


... recursive call search

Cand = $[[n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)],$
 $[n(d, 4, 2), n(c, 1, 1), n(a, 0, 0)], [n(b, 7, 1), n(a, 0, 0)],$
 $[n(d, 8, 1), n(a, 0, 0)]]$

Exp = $[[n(c, 1, 1), n(a, 0, 0)], [n(a, 0, 0)]]$

Example – traversal

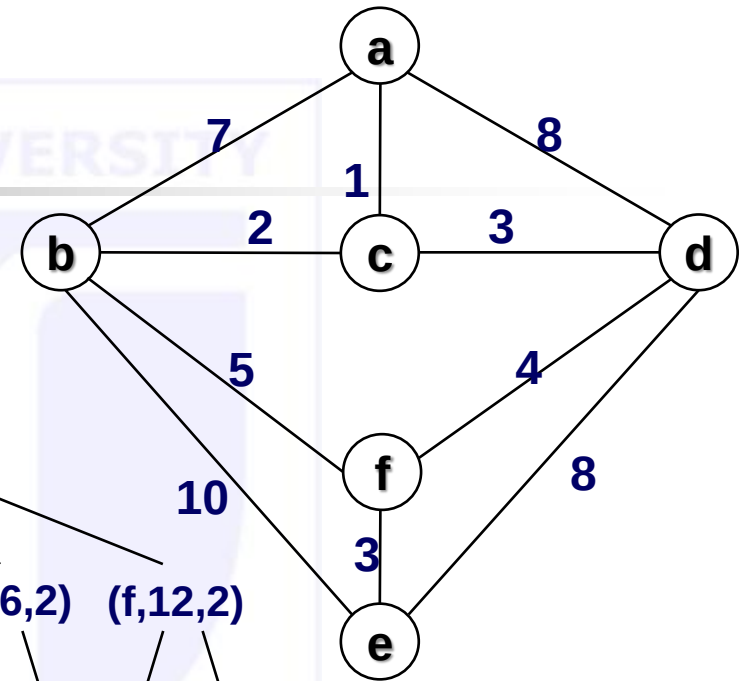
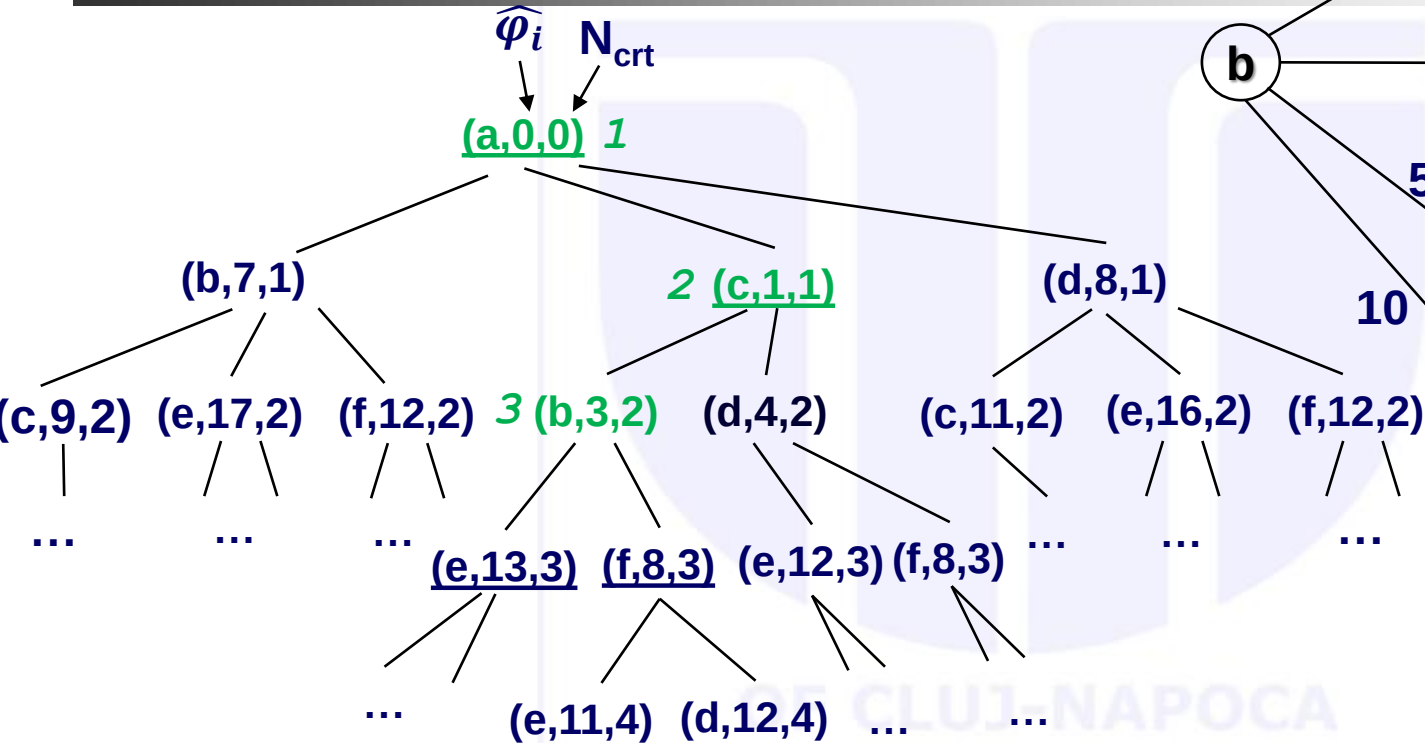


... this will be expanded next

Cand = $[[n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)],$
 $[n(d, 4, 2), n(c, 1, 1), n(a, 0, 0)], [n(b, 7, 1), n(a, 0, 0)],$
 $[n(d, 8, 1), n(a, 0, 0)]]$

Exp = $[[n(c, 1, 1), n(a, 0, 0)], [n(a, 0, 0)]]$

Example – traversal

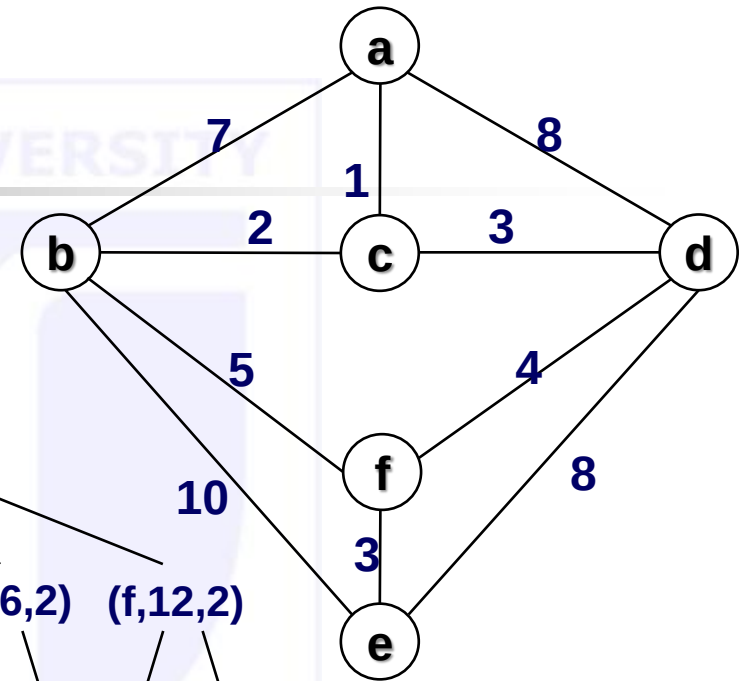
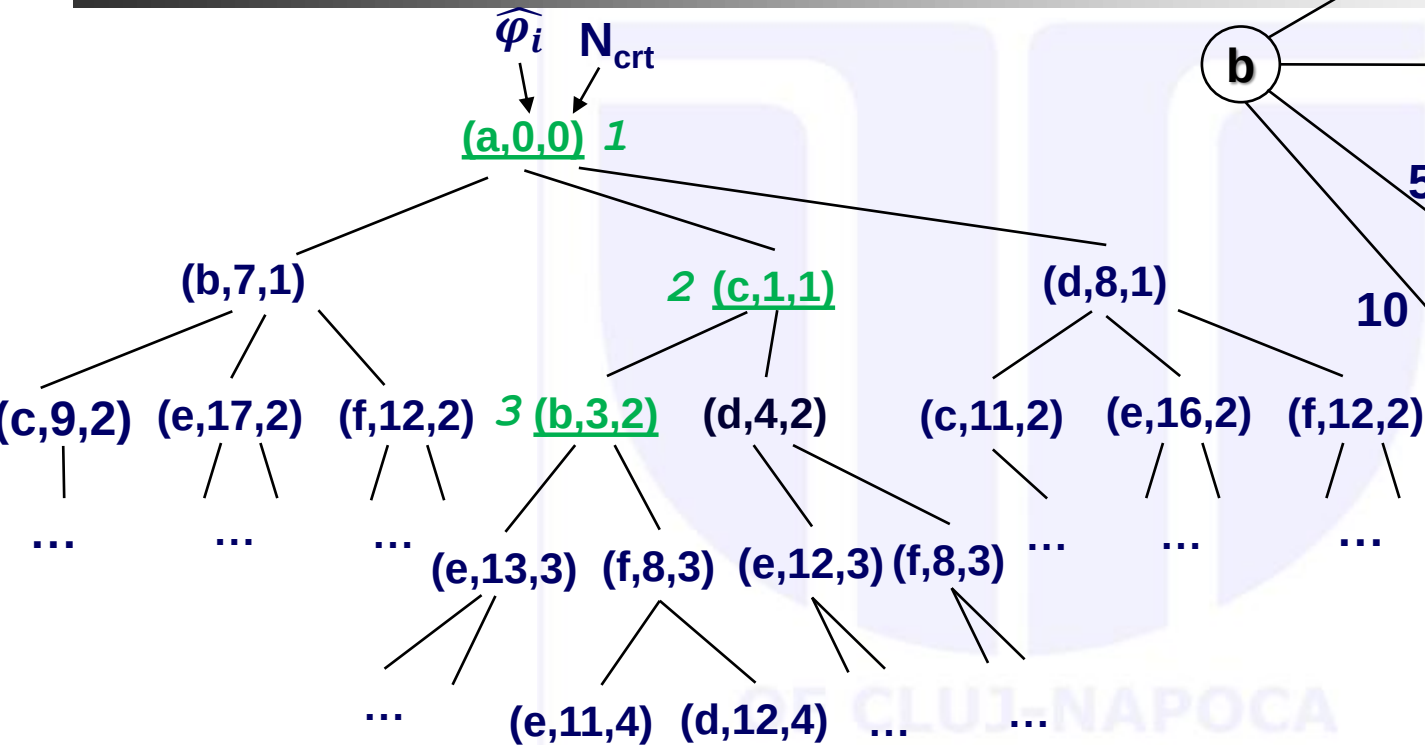


... after expanding $[n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)]$

Cand = $[[n(d, 4, 2), n(c, 1, 1), n(a, 0, 0)], [n(b, 7, 1), n(a, 0, 0)],$
 $[n(d, 8, 1), n(a, 0, 0)], [n(f, 8, 3), n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)],$
 $[n(e, 13, 3), n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)]]$

Exp = $[[n(c, 1, 1), n(a, 0, 0)], [n(a, 0, 0)]]$

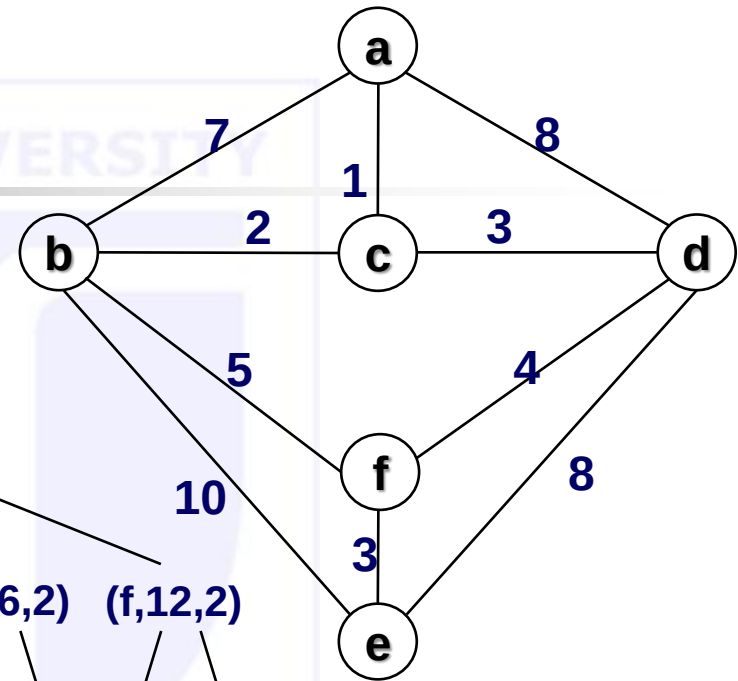
Example – traversal



... recursive call search

Cand = $[[n(d, 4, 2), n(c, 1, 1), n(a, 0, 0)], [n(b, 7, 1), n(a, 0, 0)],$
 $[n(d, 8, 1), n(a, 0, 0)], [n(f, 8, 3), n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)],$
 $[n(e, 13, 3), n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)]]$

Exp = $[[n(b, 3, 2), n(c, 1, 1), n(a, 0, 0)], [n(c, 1, 1), n(a, 0, 0)],$
 $[n(a, 0, 0)]]$

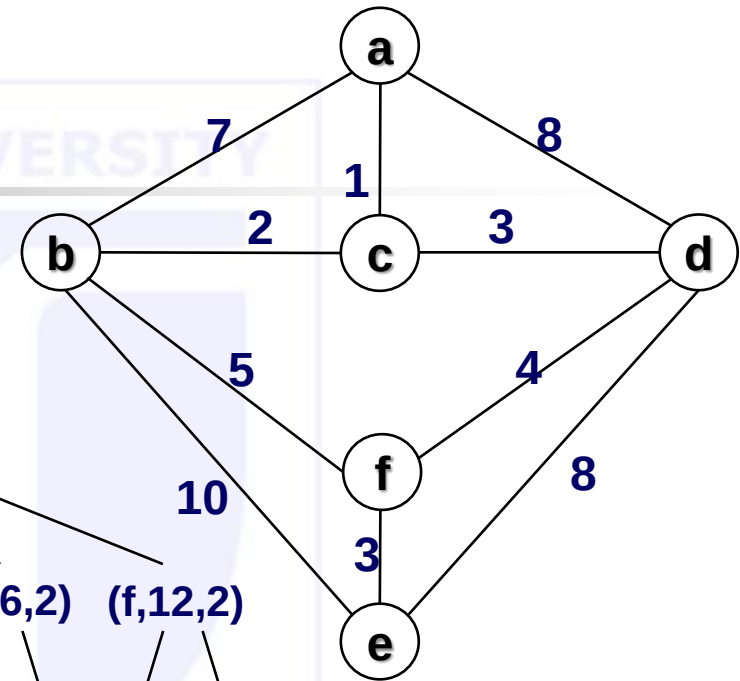
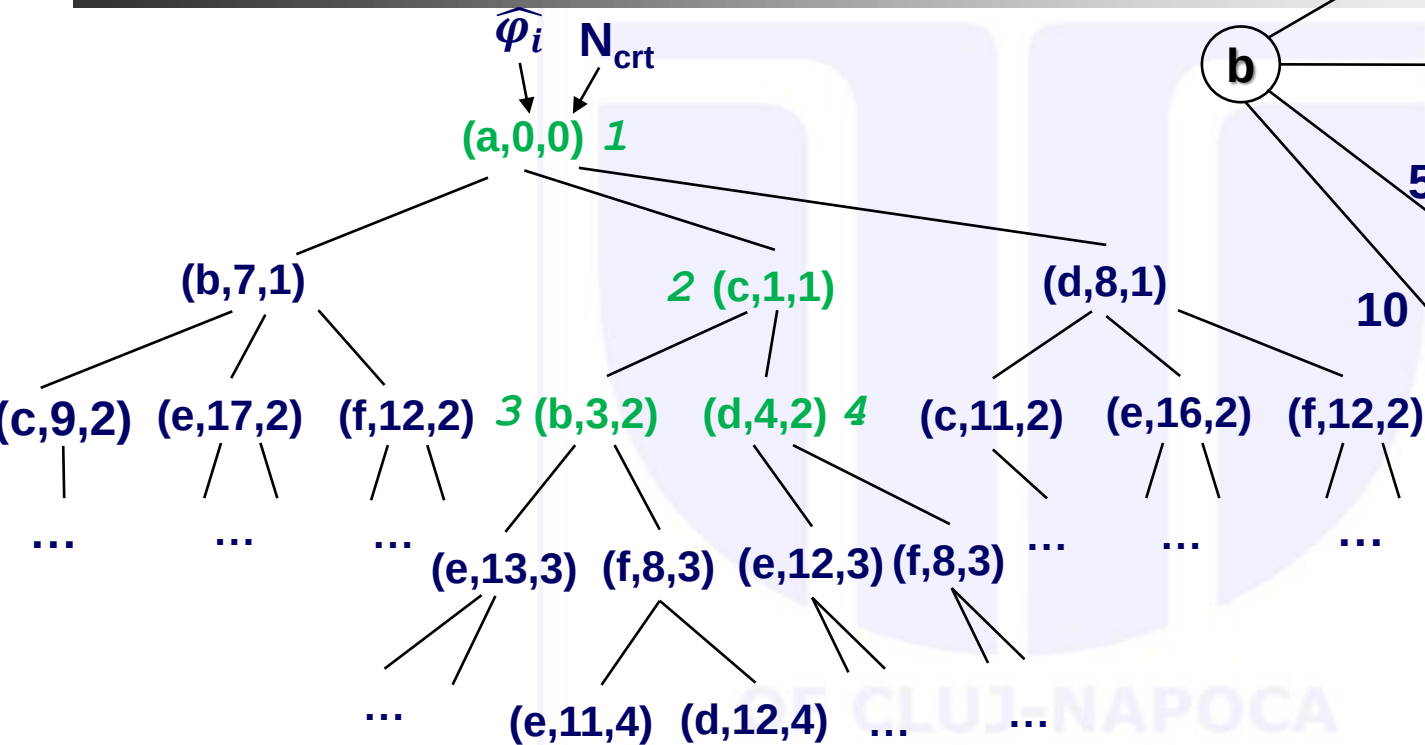


```

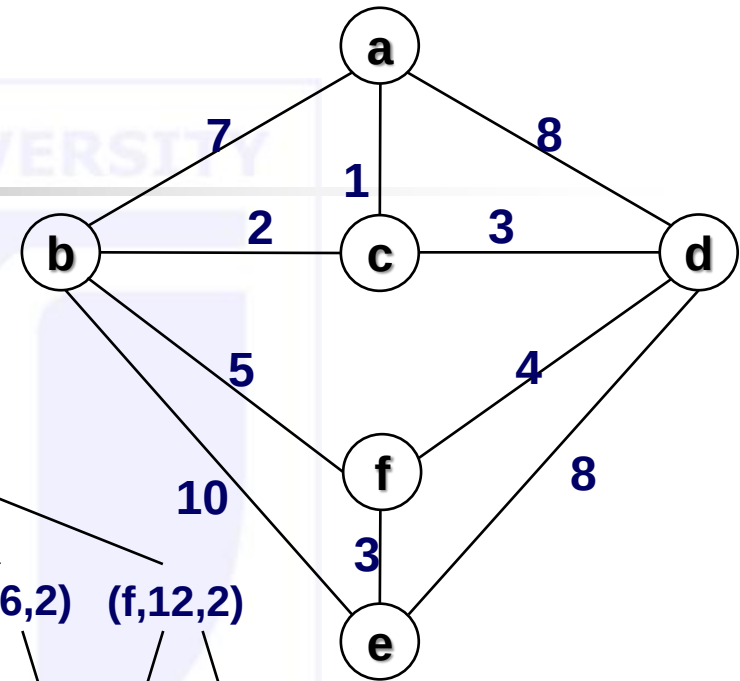
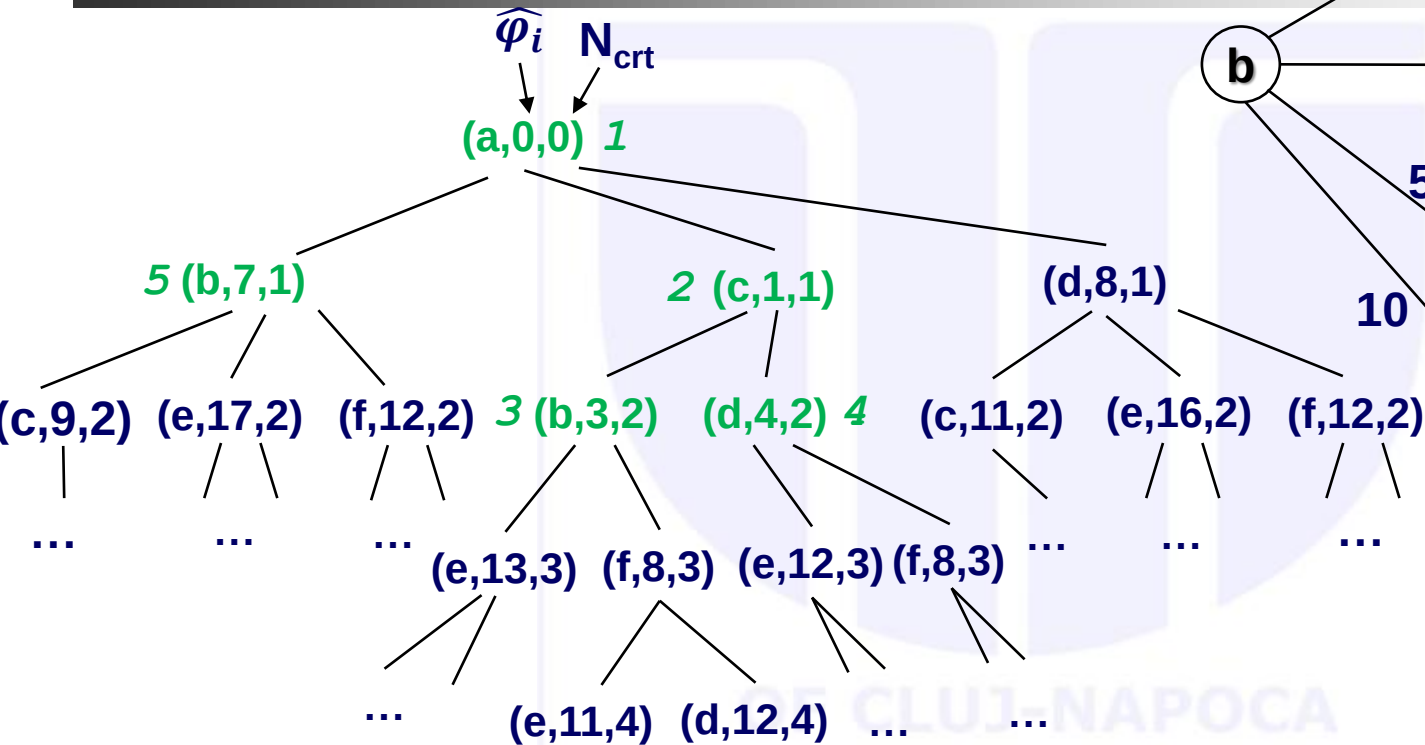
Cand      = [[n(d,4,2),n(c,1,1),n(a,0,0)], [n(b,7,1),n(a,0,0)],
              [n(d,8,1),n(a,0,0)], [n(f,8,3),n(b,3,2),n(c,1,1),n(a,0,0)],
              [n(e,13,3),n(b,3,2),n(c,1,1),n(a,0,0)]]
Exp       = [[n(b,3,2),n(c,1,1),n(a,0,0)], [n(c,1,1),n(a,0,0)],
              [n(a,0,0)]]

```

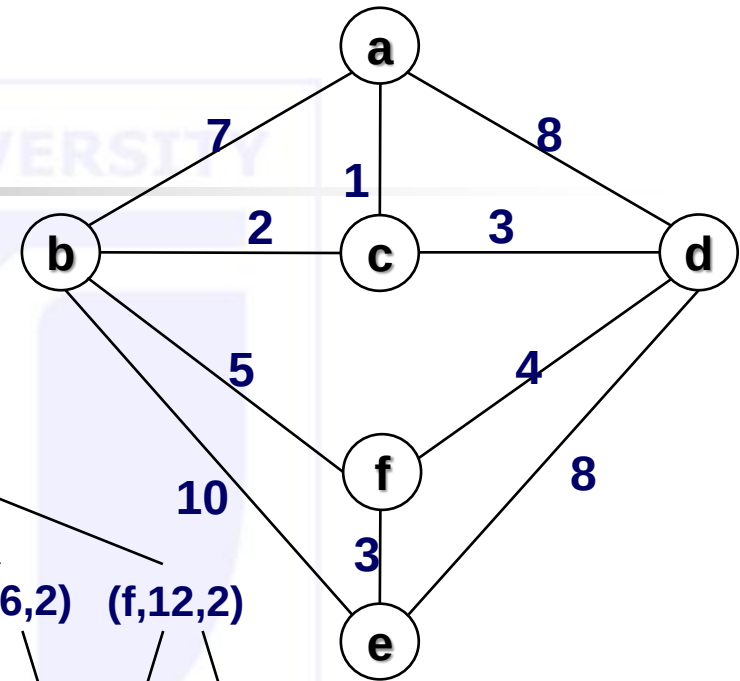
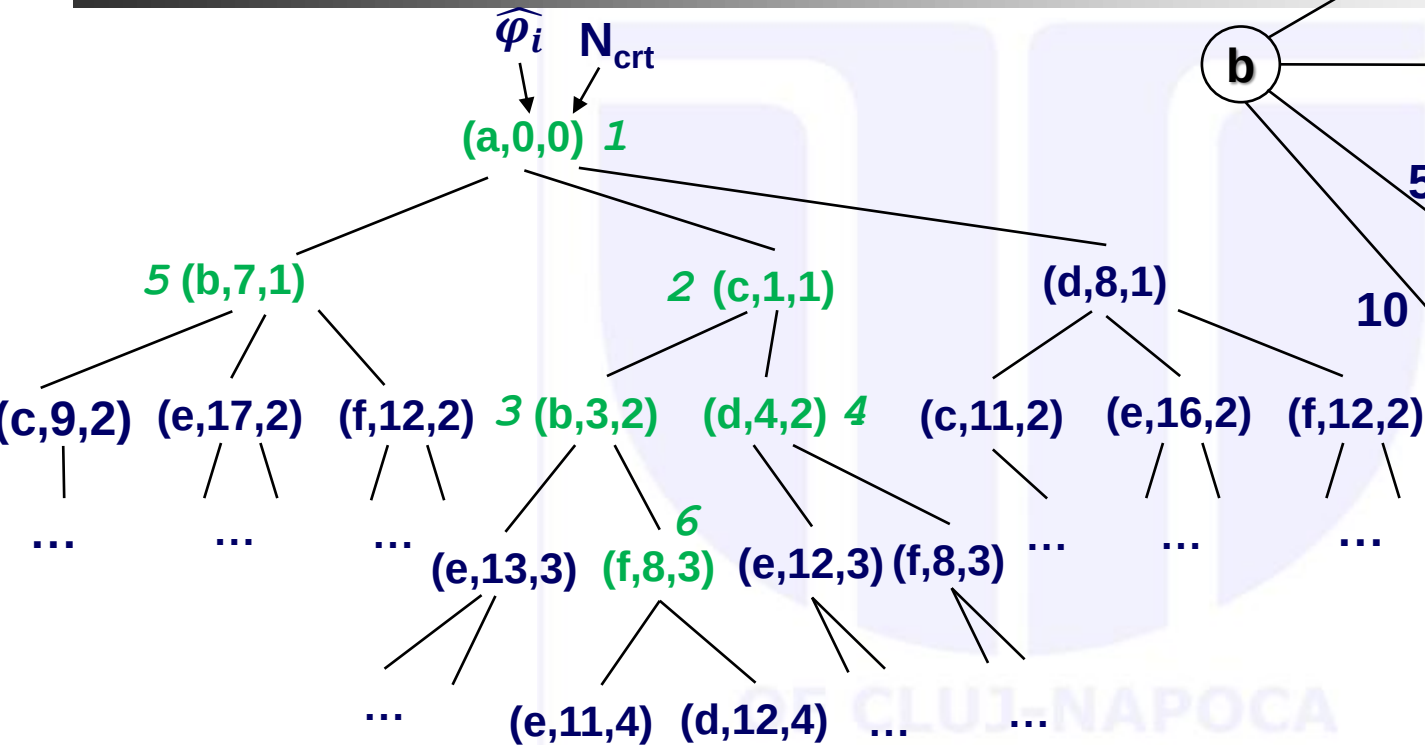
Example – traversal



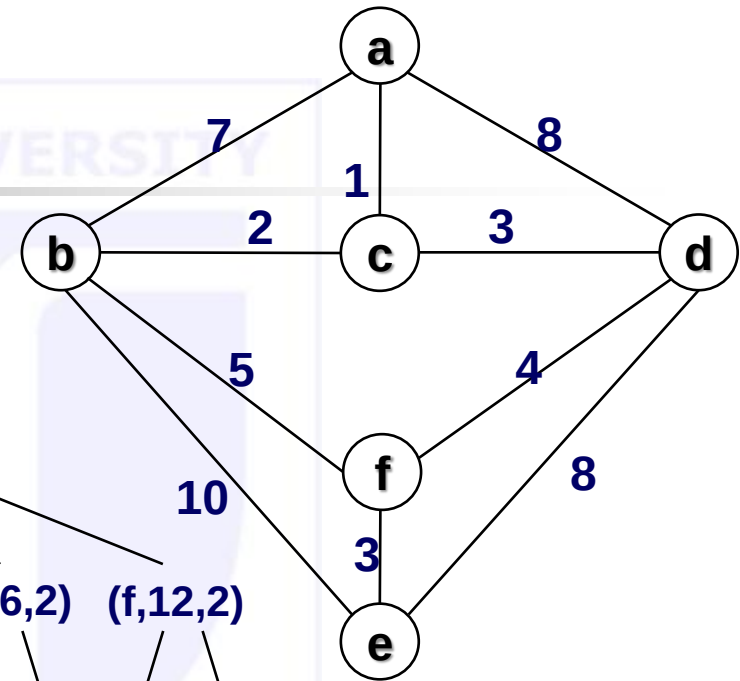
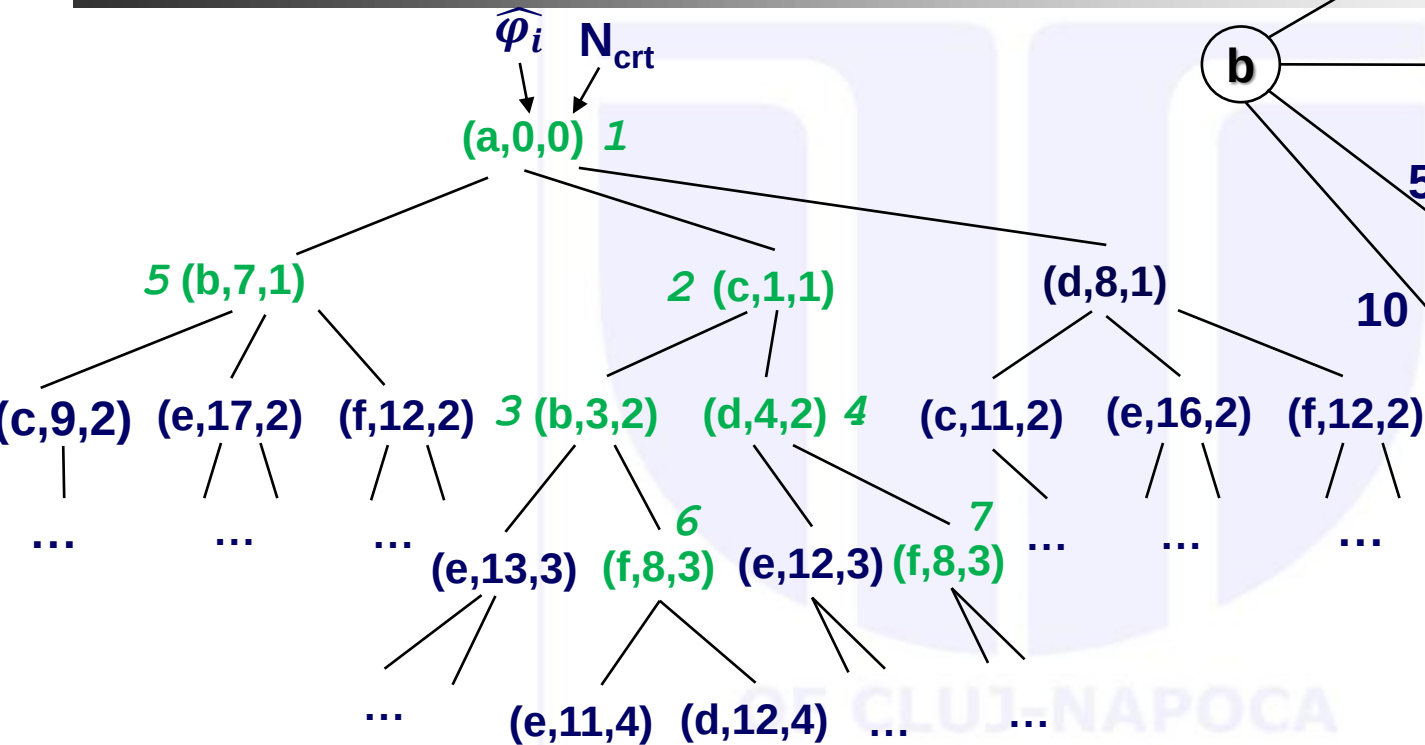
Example – traversal



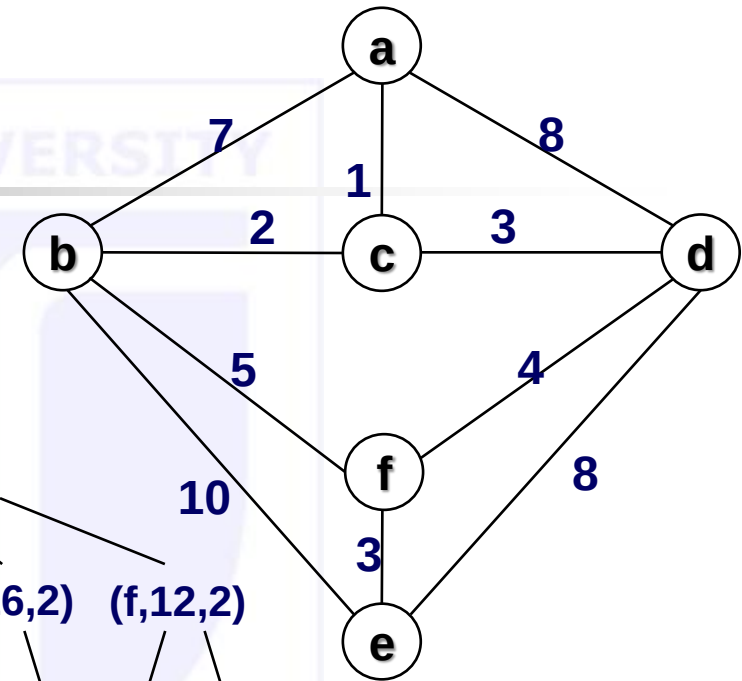
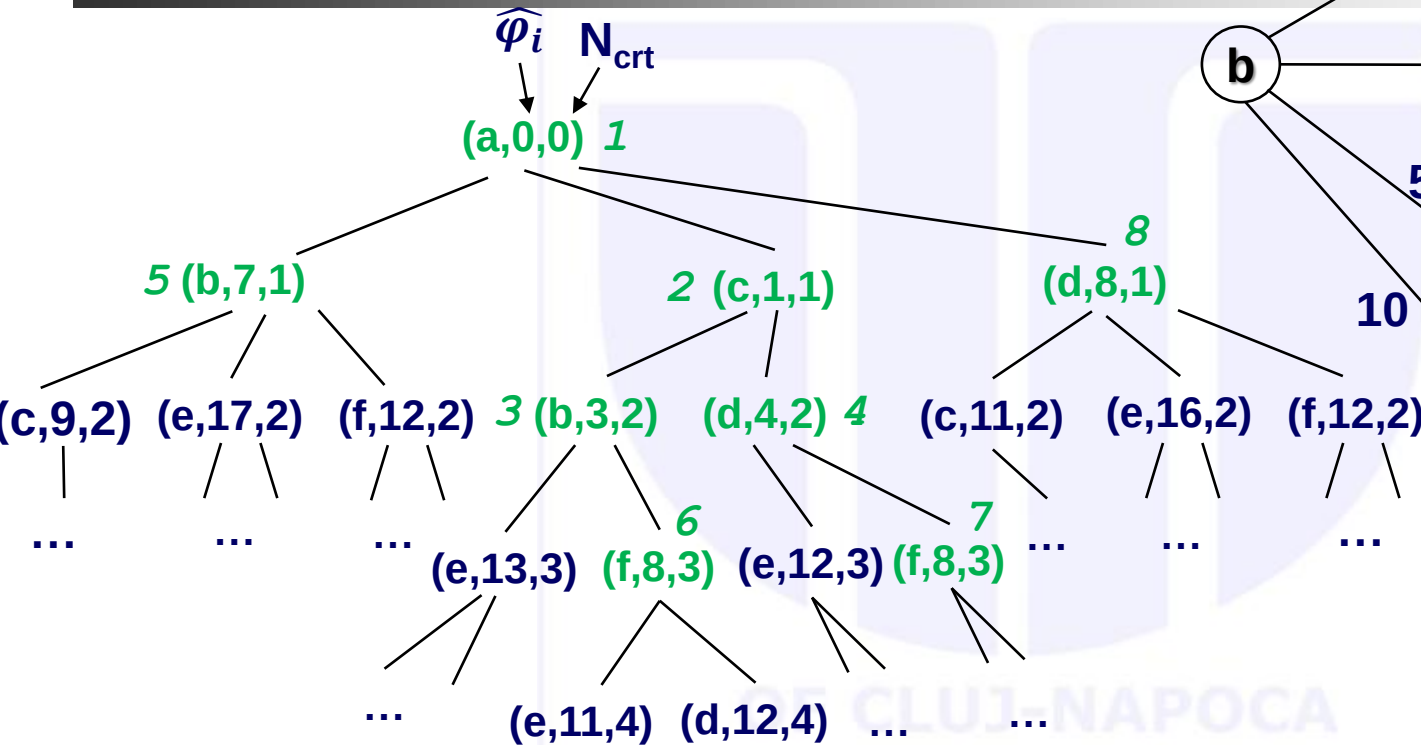
Example – traversal



Example – traversal



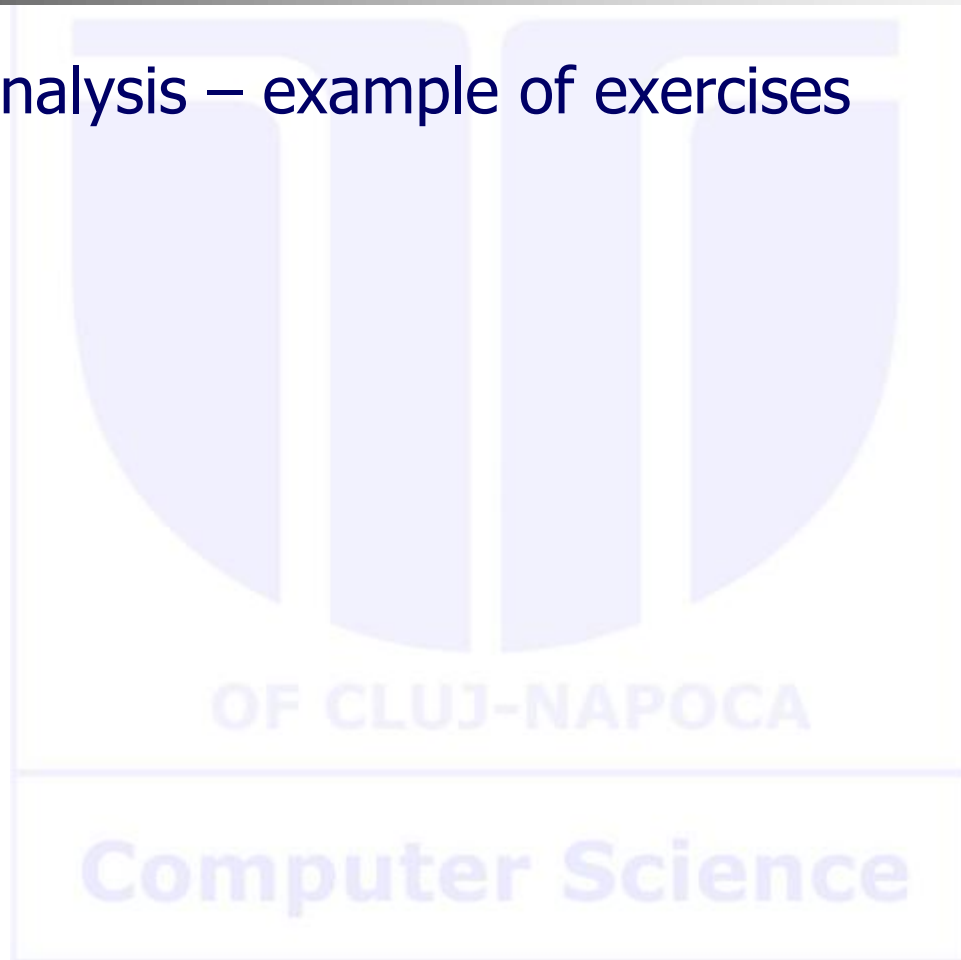
Example – traversal



... etc.

Your question NOW or next lecture

- Some Quiz analysis – example of exercises



Quiz example

Given the compound term $n(c(A, B), v, m(1, [2, 3]))$, which of the following terms it CANNOT unify with, and why?

Select one:

- ☐ a. $N(c(a, B), v, m(1, [2, 3]))$, because the name of the structure has to be constant
- ☐ b. $n(A, B, C)$, because not all three arguments can be variable
- ☐ c. $N(c(a, B), v, m(1, [H|T]))$, because the third arguments do not unify
- ☐ d. $n(c(a, B), v, m(1, [2, B]))$, because the unifications involving B are in contradiction

Quiz example – contd.

Choose the right implementation of the `member` predicate to ensure a nondeterministic behavior:

Select one or more:

- ☐ a. `member(H, [H|_]) .`
`member(H, [_|T]) :-`
`member(H, T) .`
- ☐ b. `member(H, [H|_]) :- ! .`
`member(H, [_|T]) :-`
`member(H, T) .`
- ☐ c. `member(H, [H|_]) .`
`member(H, [_|T]) :- ! ,`
`member(H, T) .`
- ☐ d. `member(_, []) .`
`member(H, [H|_]) .`
`member(H, [_|T]) :-`
`member(H, T) .`

Quiz example – contd.

The predicate below is assumed to remove **just the first** occurrence of the first argument, from the list given in the second argument, to generate the list in the third argument.

To behave accordingly (as in the associated example), the code should be updated in the following way:

```
delete(X, [X|T], T) .  
  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .  
  
delete(_, [], []) .  
  
?- delete(b, [a,b,c,b,d], R) .  
  
R= [a,c,b,d]? ;  
  
no
```

Select one:

- ☐ a. Update the clause 1 as `delete(X,[X|T],T):-!`.
- ☐ b. Remove the clause `delete(X,[X|T],T)`.
- ☐ c. Add the clause: `delete(X,[X],[])`.
- ☐ d. Remove the clause `delete(_,[],[])`.
- ☐ e. Swap the order of clauses 1 and 3.
- ☐ f. Update the clause 2 as `delete(X,[H|T],[H|R]):-!,delete(X,T,R)`.
- ☐ g. Swap the order of clauses 1 and 2.
- ☐ h. Update the clause 2 as `delete(X,[H|T], R):-!,delete(X,T,R)`.

Quiz example – contd.

The code below is assumed to collect in **IN**order the keys from a BST. Choose the best fit statement among the options:

```
tree_2_list_in(nil,_,_).  
tree_2_list_in(t(Left,Key,Right),First,Last):-  
    tree_2_list_in(Right,Int,Last),  
    tree_2_list_in(Left,First,[Key|Int]).
```

Select one:

- ☐ a. The code is correct as it is if and only if the initial call sets the last argument to the empty list.
- ☐ b. To be correct, the order of the recursive calls must be swapped.
- ☐ c. The code is correct as it is.
- ☐ d. To be correct, in the first clause arguments 2 and 3 should share the same variable.

Quiz example – contd.

Given the edge representation of an undirected graph, the predicate `difference_graph` below generates the difference graph Gdiff of G1 and G2 (assume a predicate `is_edge` is present for each 3 graphs - G1 G2 and Gdiff):

```
difference_graph:-
    is_edge_1(X,Y),
    not(is_edge_2(X,Y)),
    not(is_edge_diff(X,Y)),
    assertz(edge_diff(X,Y)),
    fail.
difference_graph.

is_edge(X,Y):-
    edge(X,Y);
    edge(Y,X).
```

Choose the most specific correct answer from below:

Select one:

- ☐ a. The sequence is correct as both G1 and G2 are entirely scanned once.
- ☐ b. The sequence is incorrect, as it is missing a ! after `not(is_edge_2(X,Y))`.
- ☐ c. The sequence is correct as G1 is entirely scanned and G2 is scanned for each edge in G1.
- ☐ d. The sequence is incorrect, as it is missing a ! after `not(is_edge_diff(X,Y))`.

Summary / Cheat Sheet

% Template for complete list operations with backward recursion

```
pred([], "neutral_element").
pred([H|T], R):- pred(T, R1), R "=" R1 "op" H.
```

% Template for complete list operations with forward recursion

```
pred([], R, R).
pred([H|T], Acc, R):- Acc1 "=" Acc "op" H, pred(T, Acc1, R).
```

```
pred(L, R):- pred(L, "neutral_element", R).
```

% Template for complete list conditional operations

```
pred([], "neutral_element").
pred([H|T], R):- cond(...), !, pred(T, R1), R "=" R1 "op" H.
pred([H|T], R):- pred(T, R1), R "=" R1 "op".
```

% Template for deep list operations

```
pred([], "neutral_element").
pred([H|T], R):- atomic(H), !, pred(T, R1), R "=" R1 "op" H.
pred([H|T], R):- pred(H, R1), pred(T, R2), R "=" R1 "op" R2.
```

% Template for incomplete list operations

```
pred(L, "neutral_element"):-var(L),!.
pred([H|T], R):- pred(T, R1), R "=" R1 "op" H.
```

% Template for difference list (1st and 2nd arguments) operations

```
pred(S, E, "neutral_element"):-var(S),!,var(E),!.S==E,!.
pred([H|T], E, R):- pred(T, E, R1), R "=" R1 "op" H.
```

% Template for complete tree operations

```
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" K "op" NR.
pred(nil, "neutral_element").
```

% Template for complete tree conditional operations

```
pred(t(K,L,R), Result):-
    cond(...), !,
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" K "op" NR.
```

```
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" NR.
pred(nil, "neutral_element").
```

% Template for incomplete tree operations

```
pred(T, "neutral_element"):- var(T), !.
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" K "op" NR.
```

Summary / Cheat Sheet

% Templates for dynamic operations (2 stage processes)

```
:-dynamic p/1.
p(1).
...
```

% Template for failure driven loop

```
pred(_):- failure_driven_loop.
pred(L):- collect(L).
```

failure_driven_loop:-

```
    p(X), % / retract(p(X)),
    process(X),
    fail.
```

% Template for retract recursion loop

```
pred(_):- retract_recursion.
pred(L):- collect(L).
```

retract_recursion:-

```
    retract(p(X)), % / p(X), not(seen(X)), !, assert(seen(X)),
    process(X),
    retract_recursion.
```

```
process(X):- assert(q(X)).
```

```
collect([X|P]):-
    retract(visited_node(X)), !,
    collect(P).
collect([]).
```

% Template for graph path operations

```
path(X, Y, [X|Path]):- path(X, Y, [X], Path).
```

```
path(Y, Y, _, []).
```

```
path(X, Y, PPath, [Z|Path]):-
```

```
    is_edge(X, Z),
    not(member(Z, PPath)),
    path(Z, Y, [Z|PPath], Path).
```

```
edge(1,2).
```

```
% ...
```

```
is_edge(X,Y):- edge(X,Y);edge(Y,X).
```